

MVLU COLLEGE

A.I. Practical No.4

Aim:- Feed Forward Backpropagation Neural Network

1. Implement the Feed Forward Backpropagation algorithm to train a neural network.
2. Use a given dataset to train the neural network for a specific task.
3. Evaluate the performance of the trained network on test data.

Code:-

```
Practical 4.py - D:\Har\AI\Practical 4.py (3.13.5)
File Edit Format Run Options Window Help

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load IPL Most Runs dataset
print("Hariprasad Vishwakarma T127")
df = pd.read_csv(r"F:\Downloads\Most Runs - 2020.csv")
print("Dataset Loaded Successfully!")
print("\nDataset Shape:")
print(df.shape)
print("\nFirst 5 Rows:")
print(df.head())

# Remove unnecessary columns
if "Player" in df.columns:
    df = df.drop("Player", axis=1)
if "POS" in df.columns:
    df = df.drop("POS", axis=1)

# Clean Highest Score column
if "HS" in df.columns:
    df["HS"] = df["HS"].astype(str).str.replace("*", "", regex=False)
    df["HS"] = pd.to_numeric(df["HS"], errors="coerce")

# Convert columns to numeric
for column in df.columns:
    df[column] = pd.to_numeric(df[column], errors="coerce")

# Remove missing values
df = df.dropna()
print("\nMissing values removed")

# Create performance categories using Runs
def performance_category(runs):
    if runs <= 250:
        return "Low"
    elif runs < 450:
        return "Medium"
    else:
        return "High"

df["Performance_Category"] = df["Runs"].apply(performance_category)

# Separate input and output
X = df.drop(["Runs", "Performance_Category"], axis=1)
y = df["Performance_Category"]

# Convert target classes into numerical values
encoder = LabelEncoder()
y = encoder.fit_transform(y)
print("\nClasses:")
for i, name in enumerate(encoder.classes_):
    print(i, "=", name)

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("\nTraining Samples:")
print(len(X_train))
print("\nTesting Samples:")
print(len(X_test))

# Standardize input features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Feed Forward Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation="relu",
    solver="adam",
    max_iter=500,
    random_state=42
)

# Train the neural network
model.fit(X_train, y_train)
print("\nNeural Network trained successfully!")

# Prediction
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(round(accuracy * 100, 2), "%")

# Classification Report
print("\nClassification Report:")
print(classification_report(
```

Hariprasad Vishwakarma

T127

TYCS

MVLU COLLEGE

A.I. Practical No.4

```
Practical 4.py - D:/Hari/AI/Practical 4.py (3.13.5)
File Edit Format Run Options Window Help
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
print(accuracy)
print("\nAccuracy Percentage:")
print(round(accuracy * 100, 2), "%")
# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=encoder.classes_,
    zero_division=0
))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
# Display Confusion Matrix
plt.figure(figsize=(7, 6))
plt.imshow(cm)
plt.title("Confusion Matrix - IPL Player Performance")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.xticks(range(len(encoder.classes_)), encoder.classes_)
plt.yticks(range(len(encoder.classes_)), encoder.classes_)
# Display values inside matrix
for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(j, i, cm[i, j], ha="center", va="center")
plt.colorbar()
plt.show()
# Training Loss Curve
plt.figure(figsize=(8, 5))
plt.plot(model.loss_curve_)
plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid()
plt.show()
```

Output:-

```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
----- RESTART: D:/Hari/A.I/Practical 4.py -----
Hariprasad Vishwakarma T127
Dataset Loaded Successfully!

Dataset Shape:
(133, 14)

First 5 Rows:
   POS  Player  Mat  Inns  NO  Runs  ...  BF    SR   100   50   4s   6s
0    1    KL Rahul   14    14    2    670  ...  518  129.34    1    5   58   23
1    2  Shikhar Dhawan   17    17    3    618  ...  427  144.73    2    4   67   12
2    3   David Warner   16    16    2    548  ...  407  134.64    0    4   52   14
3    4  Shreyas Iyer   17    17    2    519  ...  421  123.27    0    3   40   16
4    5   Ishan Kishan   14    13    4    516  ...  354  145.76    0    4   36   30

[5 rows x 14 columns]

Missing values removed

Classes:
0 = High
1 = Low
2 = Medium

Training Samples:
106

Testing Samples:
27

Neural Network trained successfully!

Accuracy:
0.8518518518518519

Accuracy Percentage:
85.19 %

Classification Report:
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.4

```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help

4 5 Ishan Kishan 14 13 4 516 ... 354 145.76 0 4 36 30

[5 rows x 14 columns]

Missing values removed

Classes:
0 = High
1 = Low
2 = Medium

Training Samples:
106

Testing Samples:
27

Neural Network trained successfully!

Accuracy:
0.8518518518518519

Accuracy Percentage:
85.19 %

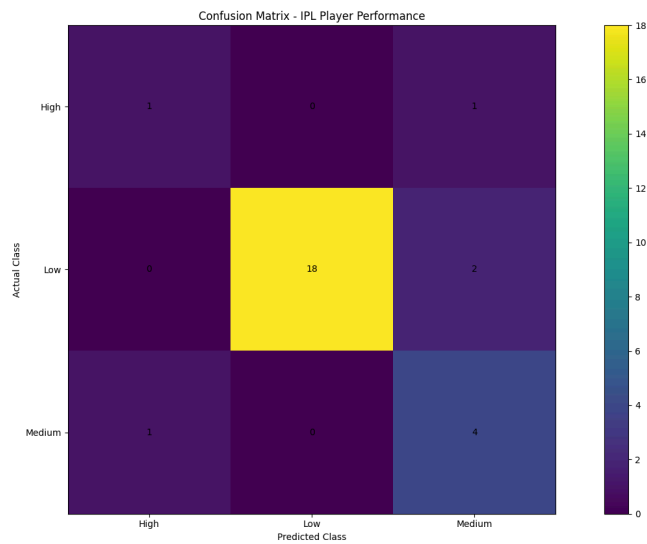
Classification Report:
              precision    recall  f1-score   support

      High       0.50      0.50      0.50         2
       Low       1.00      0.90      0.95        20
      Medium     0.57      0.80      0.67         5

   accuracy       0.85
  macro avg       0.69      0.73      0.70
weighted avg       0.88      0.85      0.86

Confusion Matrix:
[[ 1  0  1]
 [ 0 18  2]
 [ 1  0  4]]
```

Figure 1



Hariprasad Vishwakarma
T127
TYCS